

考点 11:树的基本性质

	• 小题5						• 小题5 • 应用题42	
2009	2010	2011	2012	2013	2014	2015	2016	2017
				• 小题4				
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频: 小题×3、应用题×1

树的基本性质

【考点笔记】树的基本术语

术语	定义
结点的度	树中一个结点的子结点个数。
树的度	树中结点的最大度数。
分支结点与叶子结点	度大于 0 的结点称为分支结点；度等于 0 的结点称为叶子结点。
结点的深度	从根结点开始自顶向下逐层累加。
树的高度或深度	树中结点的最大层数。
路径	树中两个结点之间所经过的结点序列。
路径长度	路径上所经过的边的条数

【考点笔记】树的基本性质

- ① 除根结点外，每个结点都有一条边指向，一条边对应一个度，因此结点总数等于总度数加 1。
- ② 度为 m 的树中有多少个结点、多少个叶子结点、分支节点？第 h 层最多有多少个结点等问题。
- ③ 森林由多棵树组成，也可从森林的角度命题。

树的基本性质

5. 若森林 F 有 15 条边、25 个结点，则 F 包含树的个数是_____。

- A. 8 B. 9 C. 10 D. 11

2016真题

42. 如果一棵非空 k ($k \geq 2$) 叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题并给出推导过程。

(1) 若 T 有 m 个非叶结点，则 T 中的叶结点有多少个？

(2) 若 T 的高度为 h (单结点的树 $h=1$)，则 T 的结点数最多为多少个？最少为多少个？

5. 在一棵度为 4 的树 T 中，若有 20 个度为 4 的结点，10 个度为 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树 T 的叶结点个数是_____。

- A. 41 B. 82 C. 113 D. 122

2010真题

结点个数、结点的度、树的高度 之间的数学关系

考点 12: 二叉树的定义与性质

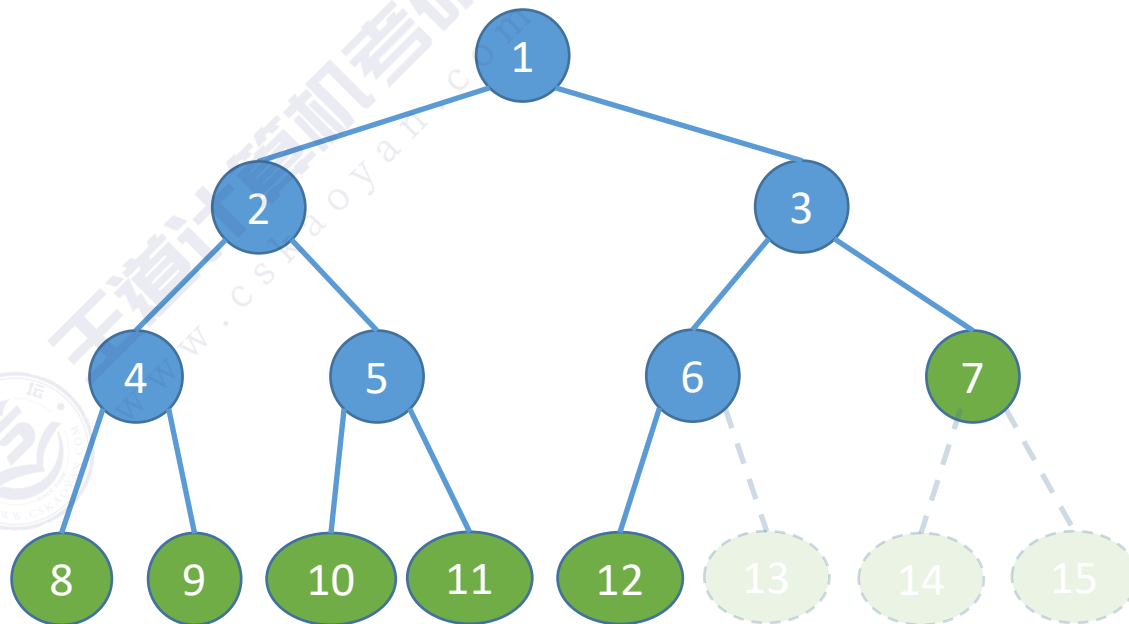
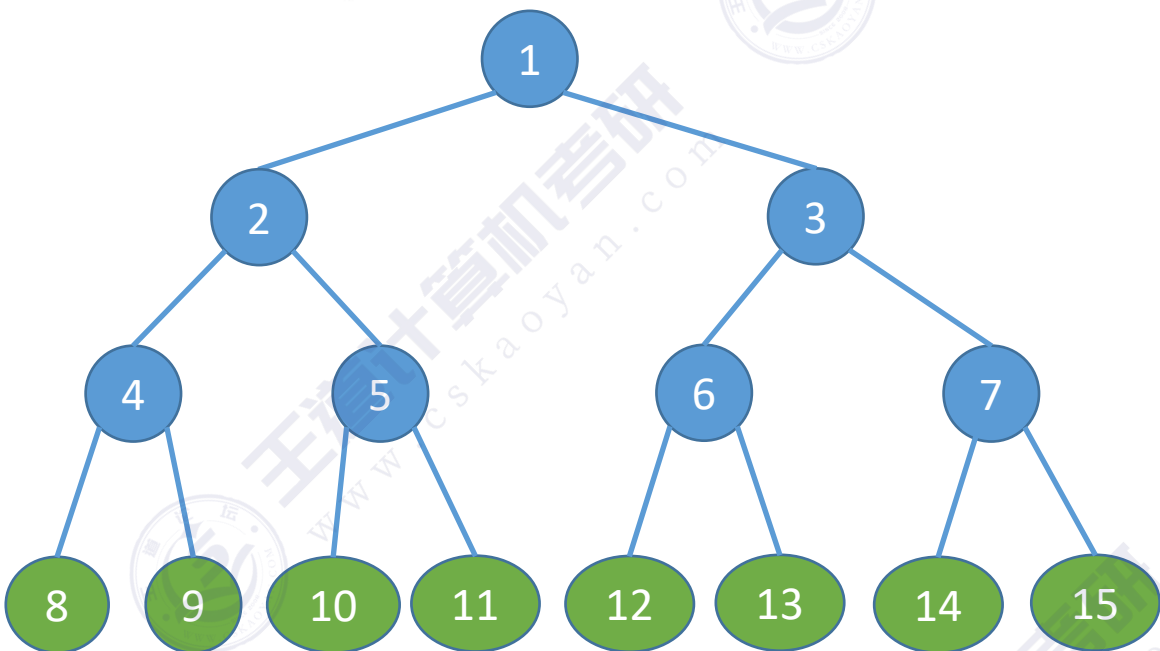
• 小题5		• 小题4						• 小题4
2009	2010	2011	2012	2013	2014	2015	2016	2017
• 小题4		• 小题3 • 应用题42		• 算法题41			• 小题3	
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频: 小题×6、算法题×1、应用题×1

二叉树的定义与性质

【考点笔记】特殊的二叉树

术语	定义
满二叉树	一棵高度为 h ，并且含有 2^h-1 个结点的二叉树称为满二叉树，即每一层的结点数都达到了其最大值（装满了！）。
完全二叉树	一棵高度为 h ，有 n 个结点的二叉树，当且仅当其每一个结点都与高度为 h 的满二叉树中编号为 $1 \sim n$ 的结点一一对应时，称为完全二叉树。 相比满二叉树，完全二叉树少了最底层、最右边的一些连续叶子结点



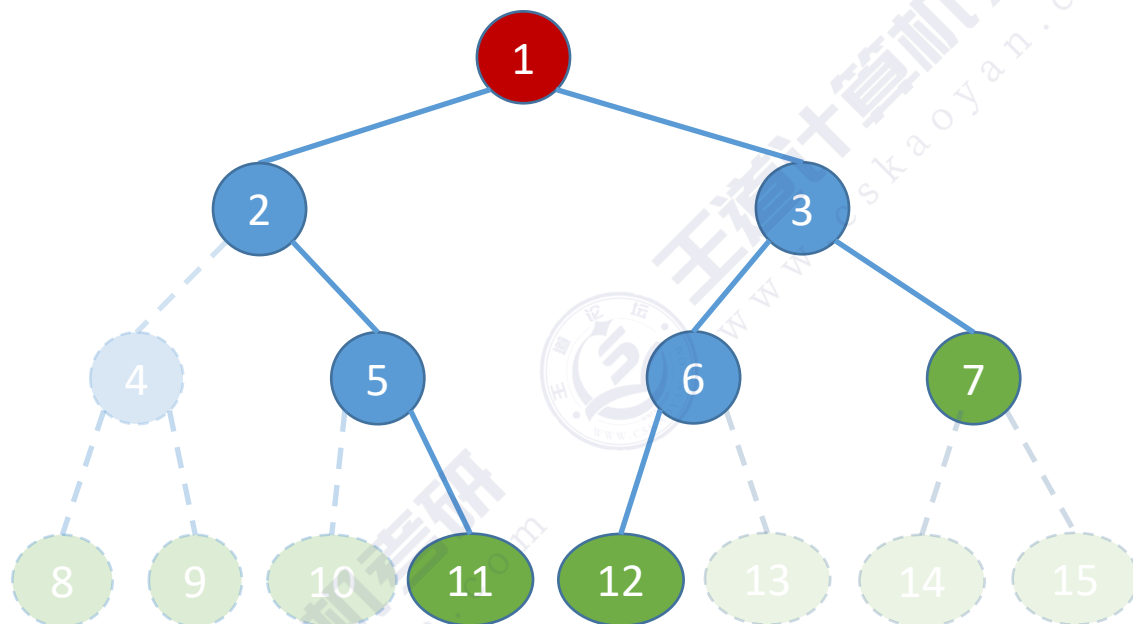
二叉树的定义与性质

【考点笔记】完全二叉树的性质

对完全二叉树按从上到下、从左到右的顺序依次编号 $1, 2, \dots, n$, 则有:

- ① 若 $i \leq \lfloor n/2 \rfloor$, 则结点 i 为分支结点, 否则为叶子结点, 也即最后一个分支结点的编号为 $\lfloor n/2 \rfloor$
(建议画几个结点数分别为 3、4、5、6 的完全二叉树检验)
- ② 叶子结点只可能在层次最大的两层上出现 (若删除满二叉树中最底层、最右边的连续 2 个或以上的叶子结点, 则倒数第二层将会出现叶子结点)。
- ③ 如果有度为 1 的结点, 只可能有一个, 且该结点只有左孩子而无右孩子 (度为 1 的分支结点只可能是最后一个分支结点, 其结点编号为 $\lfloor n/2 \rfloor$)。
- ④ 按层序编号后, 一旦出现某结点 (其编号为 i) 为叶子结点或只有左孩子, 则编号大于 i 的结点均为叶子结点 (与结论 1 和结论 3 是相通的)。
- ⑤ 若 n 为奇数, 则每个分支结点都有左子女和右子女; 若 n 为偶数, 则编号最大的分支结点 (编号为 $n/2$) 只有左子女, 没有右子女, 其余分支结点左右子女都有 (建议画几个结点数分别为 3、4、5、6 的完全二叉树检验)。
- ⑥ 结点 i 所在层次 (深度) 为 $\lfloor \log_2 i \rfloor + 1$ (与结论 7 的原理一样)。
- ⑦ 具有 n 个 ($n > 0$) 结点的完全二叉树的高度为 $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$ 。

二叉树的顺序存储



二叉树的顺序存储中，一定要把二叉树的结点编号与完全二叉树对应起来

- i 的左孩子 —— $2i$
- i 的右孩子 —— $2i+1$
- i 的父节点 —— $\lfloor i/2 \rfloor$



t[0] t[1] t[2]

二叉树的定义与性质

... 4. 设一棵非空完全二叉树 T 的所有叶结点均位于同一层，且每个非叶结点都有 2 个子结点。若 T 有 k 个叶结点，则 T 的结点总数是_____。

- A. $2k-1$ B. $2k$ C. k^2 D. 2^k-1

2018真题

... 4. 若一棵完全二叉树有 768 个结点，则该二叉树中叶结点的个数是

- A. 257 B. 258 C. 384 D. 385

2011真题

... 5. 已知一棵完全二叉树的第 6 层（设根为第 1 层）有 8 个叶结点，则该完全二叉树的结点个数最多是_____。

- A. 39 B. 52 C. 111 D. 119

2009真题

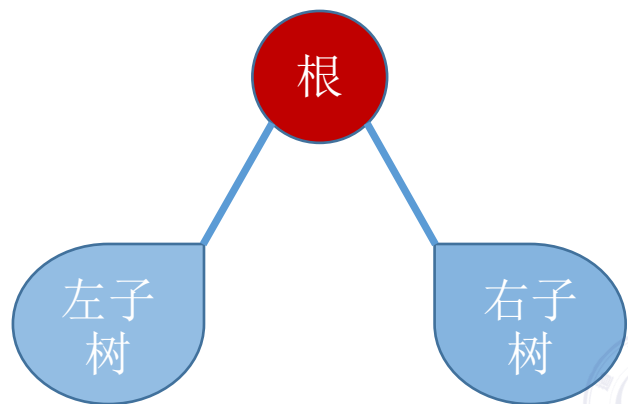
完全二叉树、满二叉树的各种性质

考点 13: 二叉树的遍历

• 小题3		• 小题5	• 小题3		• 算法题41	• 小题2 • 小题4		• 算法题41
2009	2010	2011	2012	2013	2014	2015	2016	2017
• 小题4		• 小题4 • 应用题42			• 小题3 • 算法题41	• 小题5		• 小题3
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频: 小题×10、算法题×3、应用题×1

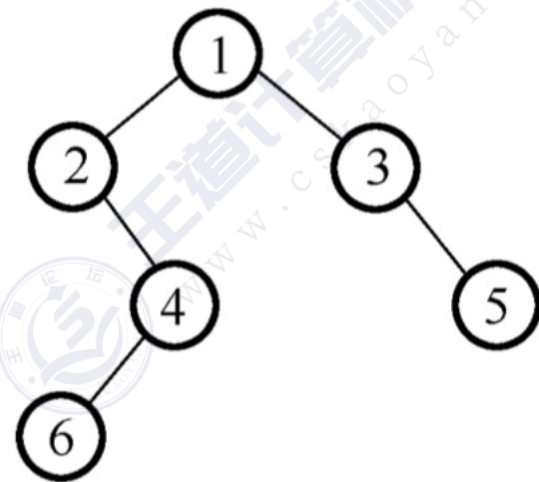
二叉树的遍历



先序遍历：根左右（NLR）

中序遍历：左根右（LNR）

后序遍历：左右根（LRN）



【考点笔记】由遍历序列构造二叉树

可以唯一确定一棵二叉树的序列组合（都需要中序序列）：

- 中序序列与前序序列
- 中序序列与后序序列
- 中序序列与层序序列

先序遍历：1 2 4 6 3 5

中序遍历：2 6 4 1 3 5

后序遍历：6 4 2 5 3 1

二叉树的遍历

4. 要使一棵非空二叉树的先序序列与中序序列相同，其所有非叶结点须满足的条件是_____。

- A. 只有左子树 B. 只有右子树 C. 结点的度均为 1 D. 结点的度均为 2

先序：根左右

中序：左根右

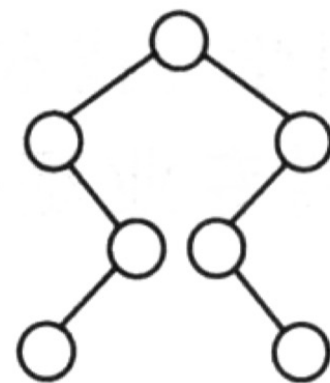
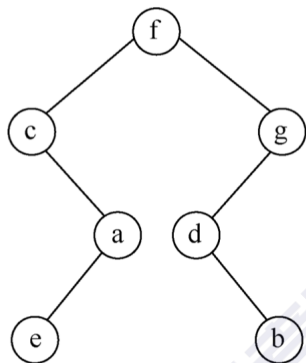
5. 已知一棵二叉树的树形如右图所示，其后序序列为 e,a,c,b,d,g,f，树中与结点 a 同层的结点是_____。

A. c

B. d

C. f

D. g

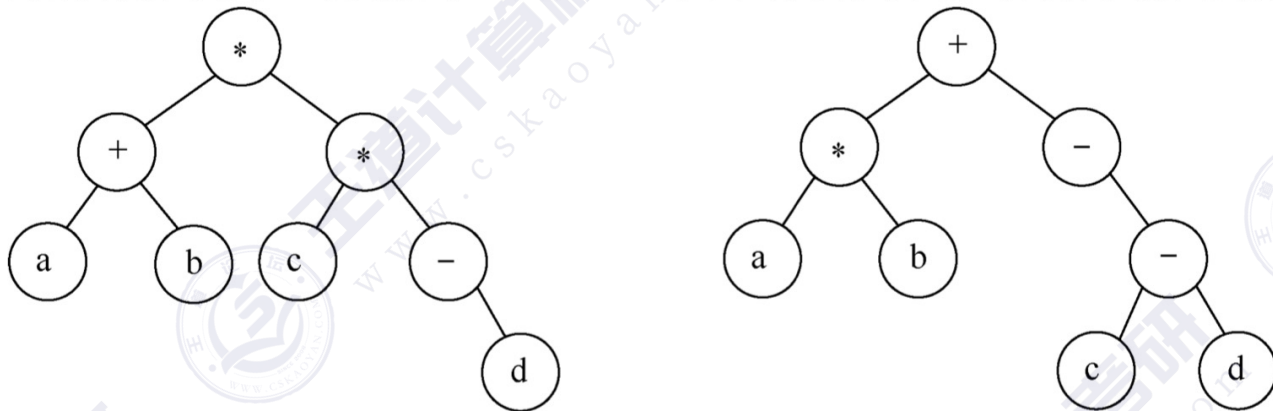


2017真题

二叉树的遍历

2017真题

41. (15 分) 请设计一个算法，将给定的表达式树（二叉树）转换为等价的中缀表达式（通过括号反映操作符的计算次序）并输出。例如，当下列两棵表达式树作为算法的输入时：



输出的等价中缀表达式分别为 $(a+b)*(c*(-d))$ 和 $(a*b)+(-(c-d))$ 。

```
typedef struct node{
    char data[10];
    struct node *left, *right;
} BTree;
```

//存储操作数或操作符

前缀表达式——前序遍历
中缀表达式——中序遍历
后缀表达式——后序遍历

要求：

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

41. (13 分) 二叉树的带权路径长度 (WPL) 是二叉树中所有叶结点的带权路径长度之和。给定一棵二叉树 T ，采用二叉链表存储，结点结构为：

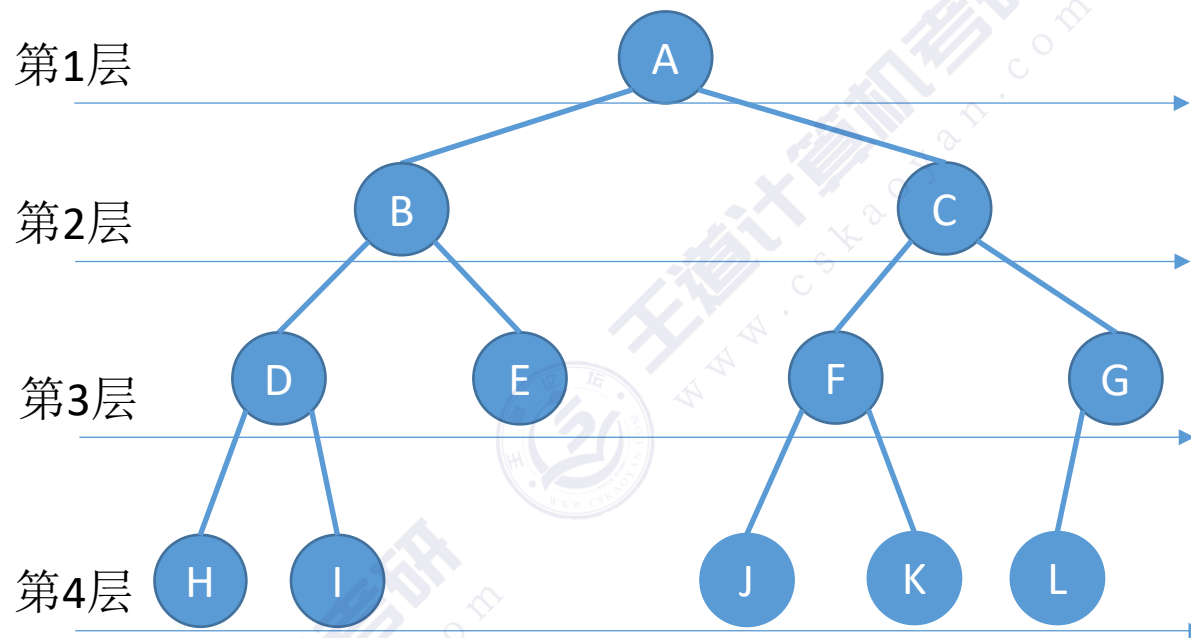
left	weight	right
------	--------	-------

其中叶结点的 `weight` 域保存该结点的非负权值。设 `root` 为指向 T 的根结点的指针，请设计求 T 的 WPL 的算法，要求：

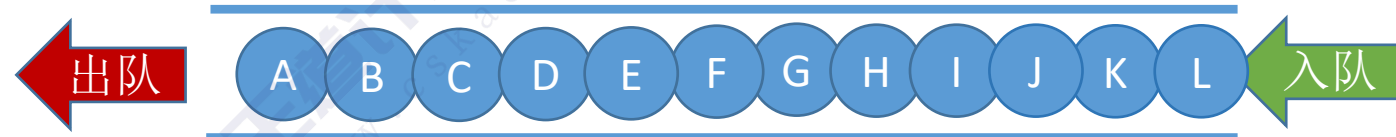
- 1) 给出算法的基本设计思想；
- 2) 使用 C 或 C++ 语言，给出二叉树结点的数据类型定义；
- 3) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

思考：结点遍历时，如何判断深度？（怎么用代码实现）

二叉树的层序遍历



注：408目前暂时没考过



算法思想：

- ①初始化一个辅助**队列**
- ②根结点入队
- ③若队列非空，则队头结点出队，访问该结点，并将其左、右孩子插入队尾（如果有的话）
- ④重复③直至队列为空

考点 14:树、森林与二叉树的转换

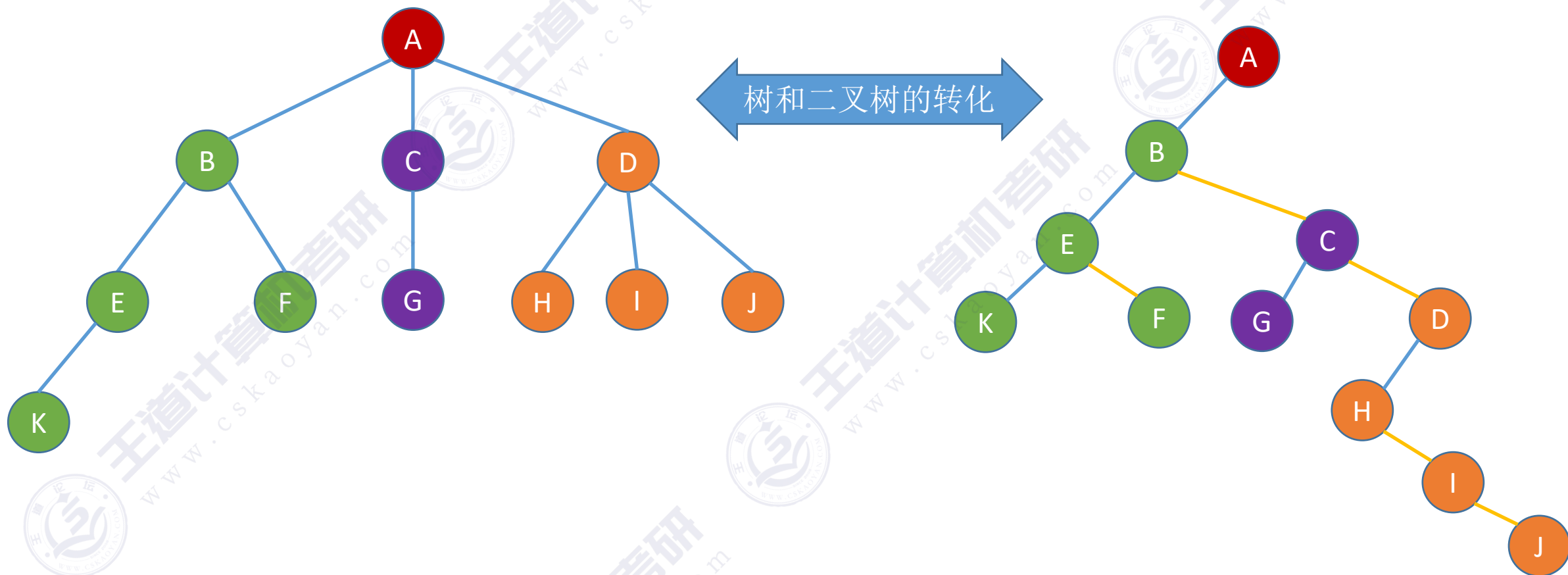
• 小题6		• 小题6			• 小题5			
2009	2010	2011	2012	2013	2014	2015	2016	2017
		• 小题2	• 小题4	• 小题4				• 小题4
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频: 小题×7

树、森林与二叉树的转换

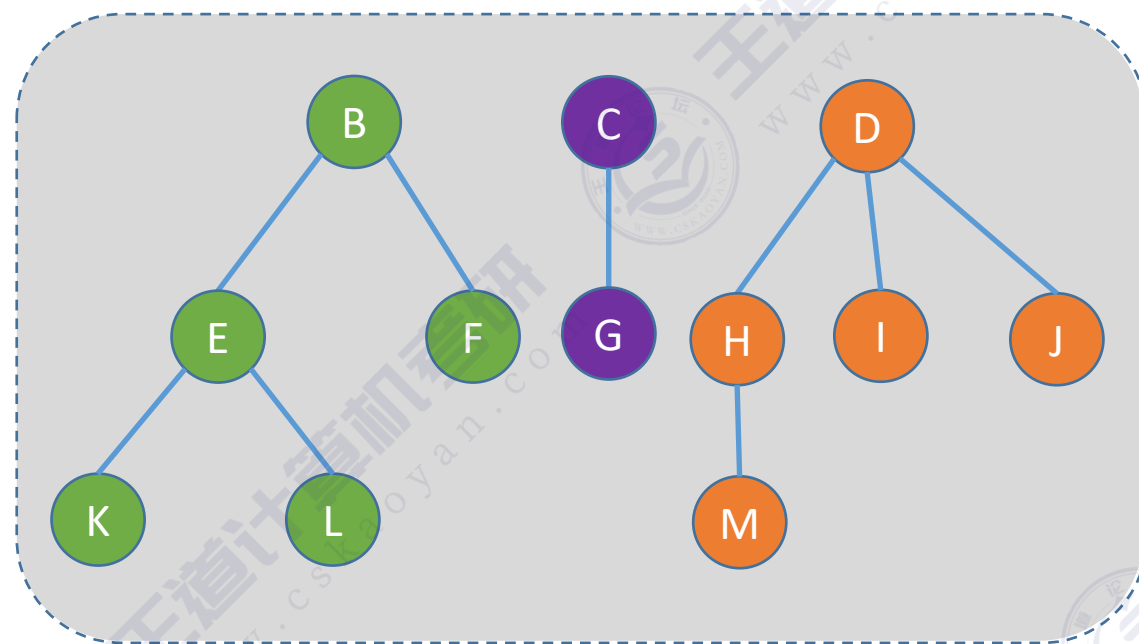
【考点笔记】树、森林与二叉树的转换

树转换为二叉树的规则：每个结点左指针指向它的第一个孩子结点，右指针指向它在树中的相邻兄弟结点。根结点没有兄弟，因此树转换而得的二叉树没有右子树。

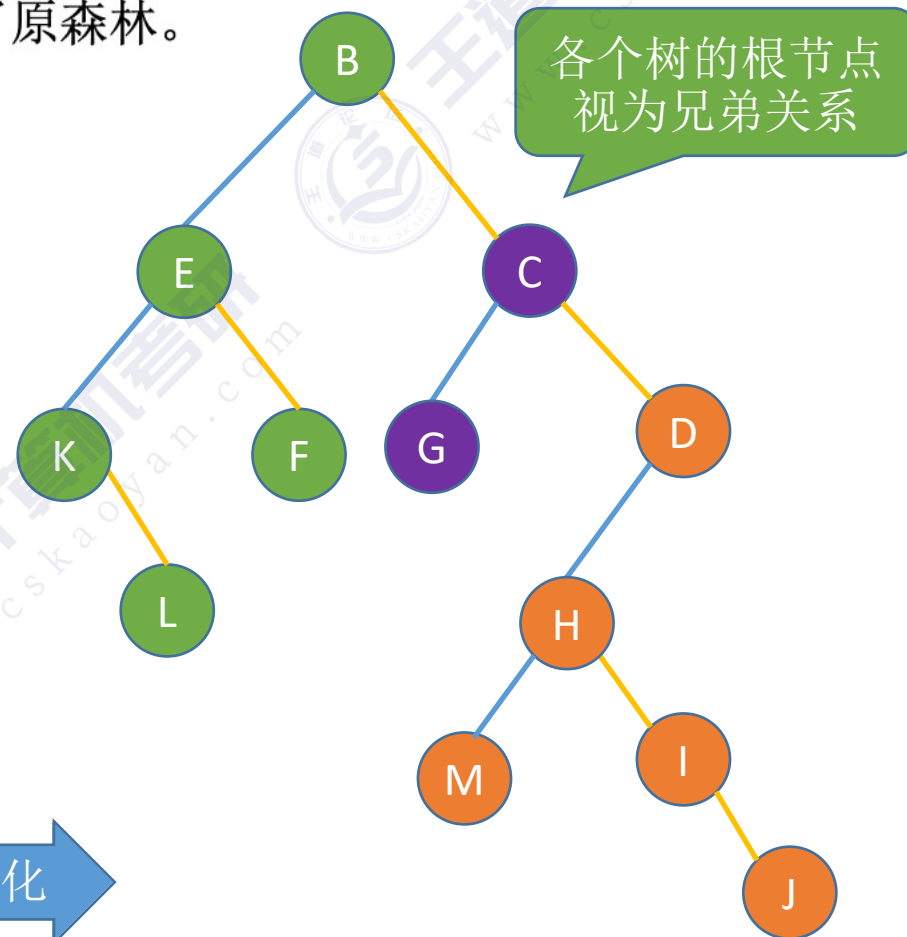


树、森林与二叉树的转换

二叉树转换为森林的规则：若二叉树非空，则二叉树根及其左子树为第一棵树的二叉树形式，二叉树根的右子树又可以看做是一个由除第一棵树外的森林转换后的二叉树，应用同样的方法，直到最后产生一棵没有右子树的二叉树为止，这样就得到了原森林。



森林和二叉树的转化



树、森林与二叉树的遍历

树	森林	二叉树
先根遍历	先序遍历（先根遍历）	先序遍历
后根遍历	中序遍历（后根遍历）	中序遍历

树、森林用孩子兄弟表示法
即可呈现出二叉树的形态

树、森林与二叉树的转换

已知森林 F 及与之对应的二叉树 T , 若 F 的先根遍历序列是 a, b, c, d, e, f , 后根遍历序列是 b, a, d, f, e, c , 则 T 的后序遍历序列是

- A. b, a, d, f, e, c
- B. b, d, f, e, c, a
- C. b, f, e, d, c, a
- D. f, e, d, c, b, a

2020真题

2. 若将一棵树 T 转化为对应的二叉树 BT , 则下列对 BT 的遍历中, 其遍历序列与 T 的后根遍历序列相同的是

- A. 先序遍历
- B. 中序遍历
- C. 后序遍历
- D. 按层遍历

2019真题

5. 将森林 F 转换为对应的二叉树 T , F 中叶结点的个数等于_____。

- A. T 中叶结点的个数
- B. T 中度为 1 的结点个数
- C. T 中左孩子指针为空的结点个数
- D. T 中右孩子指针为空的结点个数

2014真题

6. 已知一棵有 2011 个结点的树, 其叶结点个数为 116, 该树对应的二叉树中无右孩子的结点个数为_____。

- A. 115
- B. 116
- C. 1895
- D. 1896

2011真题

考点 15:线索二叉树的基本概念和构造

	• 小题3			• 小题5	• 小题4			
2009	2010	2011	2012	2013	2014	2015	2016	2017
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频: 小题×3

线索二叉树

在二叉树线索化时，通常规定：若无左子树，令 lchild 指向其前驱结点；若无右子树，令 rchild 指向其后继结点。如图 4-11 所示，还需要增加两个标志域表明当前指针域所指对象是指向左（右）子结点还是直接前驱（后继）。

ltag	lchild	data	rchild	rtag
------	--------	------	--------	------

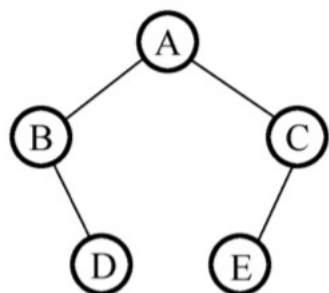
图 4-11 线索二叉树的结点结构

其中标志域含义如下：

$$\begin{aligned} \text{ltag} & \begin{cases} 0 & \text{lchild域指示结点的左孩子} \\ 1 & \text{lchild域指示结点的前驱} \end{cases} \\ \text{rtag} & \begin{cases} 0 & \text{rchild域指示结点的右孩子} \\ 1 & \text{rchild域指示结点的后继} \end{cases} \end{aligned}$$

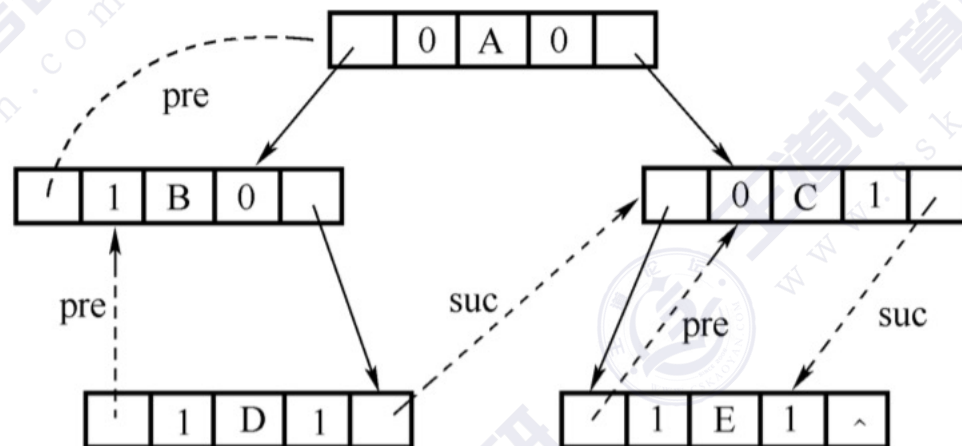
作用：方便二叉树的遍历，通过线索找前驱/后继（前、中、后序各不相同）

线索二叉树

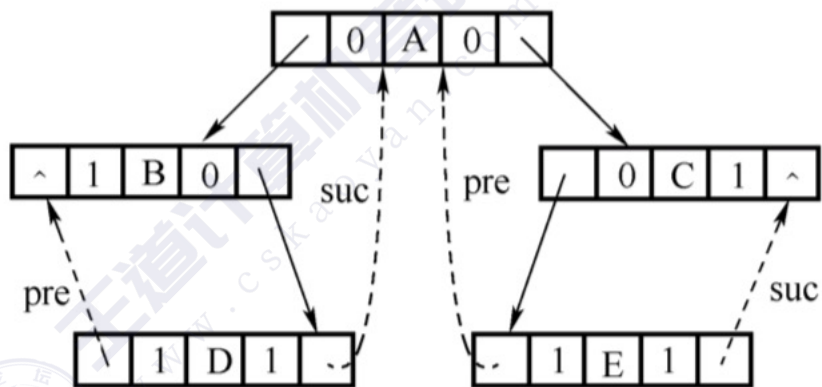


(a) 某二叉树

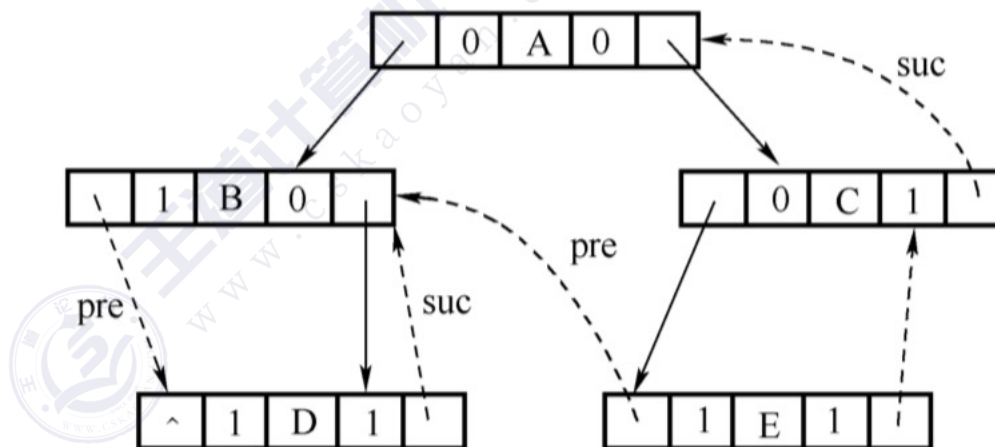
前序遍历: ABDCE
中序遍历: BDAEC
后序遍历: DBECA



(b) 前序线索树



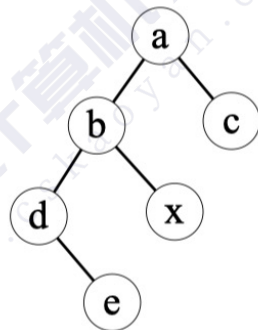
(c) 中序线索树



(d) 后序线索树

线索二叉树

4. 若对如下的二叉树进行中序线索化，则结点 x 的左、右线索指向的结点分别是_____。



A. e、c

B. e、a

C. d、c

D. b、a

2014真题

5. 若 X 是后序线索二叉树中的叶结点，且 X 存在左兄弟结点 Y。则 X 的右线索指向的是_____。

A. X 的父结点

B. 以 Y 为根的子树的最左下结点

C. X 的左兄弟结点 Y

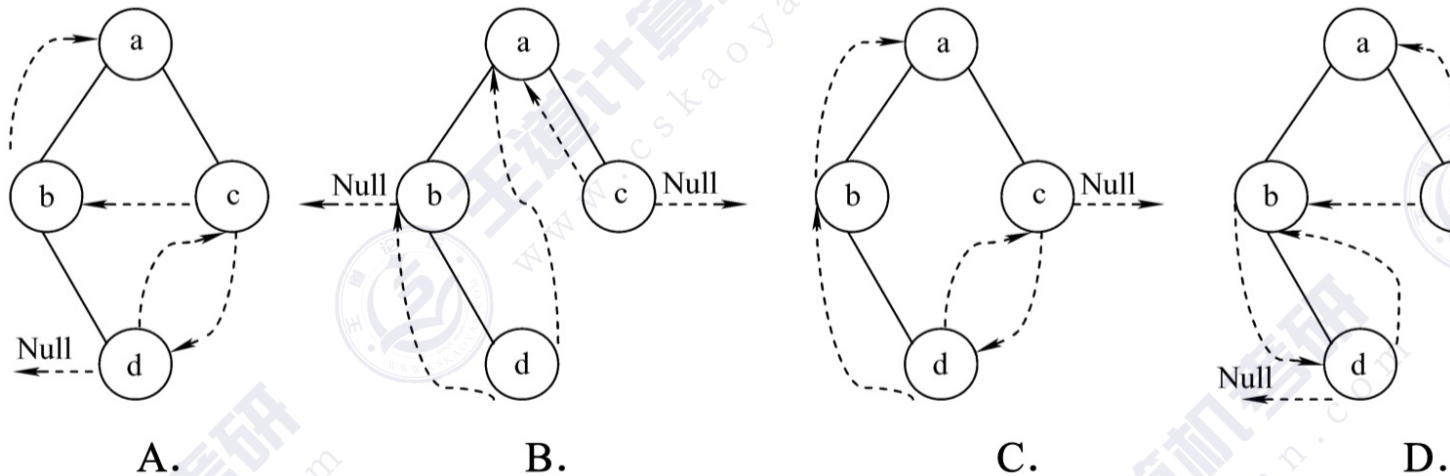
D. 以 Y 为根的子树的最右下结点

2013真题

线索二叉树

3. 下列线索二叉树中（用虚线表示线索），符合后序线索树定义的是_____。

2010真题



注：警惕考察找指定结点的前驱、找后继的问题

考点 16:哈夫曼树与哈夫曼编码

	• 小题6		• 应用题41	• 小题4	• 小题6 • 应用题41	• 小题3		• 小题6
2009	2010	2011	2012	2013	2014	2015	2016	2017
• 小题5	• 小题3	• 应用题42	• 小题5	• 小题5	• 小题4		• 小题5	
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频: 小题×11、应用题×3

哈夫曼树与哈夫曼编码

【考点笔记】哈夫曼树的定义

从树根结点到任意结点的路径长度（经过的边数）与该结点上权值的乘积称为该结点的带权路径长度。树中所有叶结点的带权路径长度之和称为该树的带权路径长度，记为

$$WPL = \sum_{i=1}^n w_i \times l_i$$

式中， w_i 是第 i 个叶结点所带的权值； l_i 是该叶结点到根结点的路径长度。

在含有 N 个带权叶子结点的二叉树中，其中带权路径长度最小的二叉树称为哈夫曼树。

... 41. (13 分) 二叉树的带权路径长度 (WPL) 是二叉树中所有叶结点的带权路径长度之和。给定一棵二叉树 T ，采用二叉链表存储，结点结构为：

left	weight	right
------	--------	-------

其中叶结点的 `weight` 域保存该结点的非负权值。设 `root` 为指向 T 的根结点的指针，请设计求 T 的 WPL 的算法，要求：

- 1) 给出算法的基本设计思想；
- 2) 使用 C 或 C++ 语言，给出二叉树结点的数据类型定义；
- 3) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

2014真题

哈夫曼树与哈夫曼编码

【考点笔记】哈夫曼树的构造

给定 N 个权值分别为 $w_1, w_2, \dots, w_N$ 的结点。通过哈夫曼算法可以构造出最优二叉树，算法的描述如下：

二叉哈夫曼树的构造

- ① 将这 N 个结点分别作为 N 棵仅含一个结点的二叉树，构成森林 F 。
- ② 构造一个新结点，并从 F 中选取两棵根结点权值最小的树作为新结点的左、右子树，并且将新结点的权值置为左、右子树上根结点的权值之和。
- ③ 从 F 中删除刚才选出的两棵树，同时将新得到的树加入 F 中。
- ④ 重复步骤 2) 和 3)，直至 F 中只剩下一棵树为止。

注意：多叉哈夫曼树的构造

2013真题

... 4. 已知三叉树 T 中 6 个叶结点的权分别是 2, 3, 4, 5, 6, 7, T 的带权（外部）路径长度最小是_____。

A. 27

B. 46

C. 54

D. 56

如果叶结点的数量不能刚好凑成一棵严格多叉哈夫曼树，则要补充权值为0的虚叶结点

严格三叉哈夫曼树中，只包含度为3、度为0的结点

$$n_3 + n_0 = 3n_3 + 1$$

$$\rightarrow n_3 = (n_0 - 1) / 2$$

$\rightarrow n_0 = 7$ 时才能除得尽

$\rightarrow$ 补充1个虚叶结点

复习：最佳归并树

哈夫曼树与哈夫曼编码

... 41. 设有 6 个有序表 A、B、C、D、E、F，分别含有 10、35、40、50、60 和 200 个数据元素，各表中元素按升序排列。要求通过 5 次两两合并，将 6 个表最终合并成 1 个升序表，并在最坏情况下比较的总次数达到最小。请回答下列问题。

- 1) 给出完整的合并过程，并求出最坏情况下比较的总次数。
- 2) 根据你的合并过程，描述 N ($N \geq 2$) 个不等长升序表的合并策略，并说明理由。

2012真题

哈夫曼树与哈夫曼编码

【考点笔记】哈夫曼树的特点

- ① 每个初始结点最终都成为叶结点，并且权值越小的结点到根结点的路径长度越大。
- ② 构造过程中共新建了 $N-1$ 个结点（双分支结点），因此哈夫曼树中结点总数为 $2N-1$ 。
- ③ 每次构造都选择 2 棵树作为新结点的孩子，因此哈夫曼树中不存在度为 1 的结点。
- ④ 用 n 个叶子结点构造的哈夫曼树形态可能不唯一，但带权路径长度唯一。

3. 对 n 个互不相同的符号进行哈夫曼编码。若生成的哈夫曼树共有 115 个结点,则 n 的值是
- A. 56 B. 57 C. 58 D. 60

2019真题

哈夫曼树与哈夫曼编码

【考点笔记】哈夫曼编码

如果没有一个编码是另一个编码的前缀，则称这样的编码为前缀编码。利用哈夫曼树可以设计出总长度最短的二进制前缀编码。

形成哈夫曼编码过程：

- ① 将每个待编码元素当做一个独立的结点，构造出对应的哈夫曼树。
- ② 从根至该字符的路径上边标记的序列，其中边标记为 0 表示“转向左孩子”，标记为 1 表示“转向右孩子”。
- ③ 从根结点到待编码元素结点的路径形成的 0、1 序列就是哈夫曼编码。

5. 已知字符集{a, b, c, d, e, f}，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。

A. 00, 1011, 01, 1010, 11, 100

B. 00, 100, 110, 000, 0010, 01

C. 10, 1011, 11, 0011, 00, 010

D. 0011, 10, 11, 0010, 01, 000

6. 5 个字符有如下 4 种编码方案，不是前缀编码的是_____。

A. 01,0000,0001,001,1

B. 011,000,001,010,1

C. 000,001,010,011,100

D. 0,100,110,1110,1100

2018真题

2014真题

哈夫曼树与哈夫曼编码

2020真题

42. (10 分) 若任一个字符的编码都不是其他字符编码的前缀, 则称这种编码具有前缀特性。现有某字符集(字符个数 ≥ 2) 的不等长编码, 每个字符的编码均为二进制的 0、1 序列, 最长为 L 位, 且具有前缀特性。请回答下列问题。

- (1) 哪种数据结构适宜保存上述具有前缀特性的不等长编码?
- (2) 基于你所设计的数据结构, 简述从 0/1 串到字符串的译码过程。
- (3) 简述判定某字符集的不等长编码是否具有前缀特性的过程。

考点 17:并查集

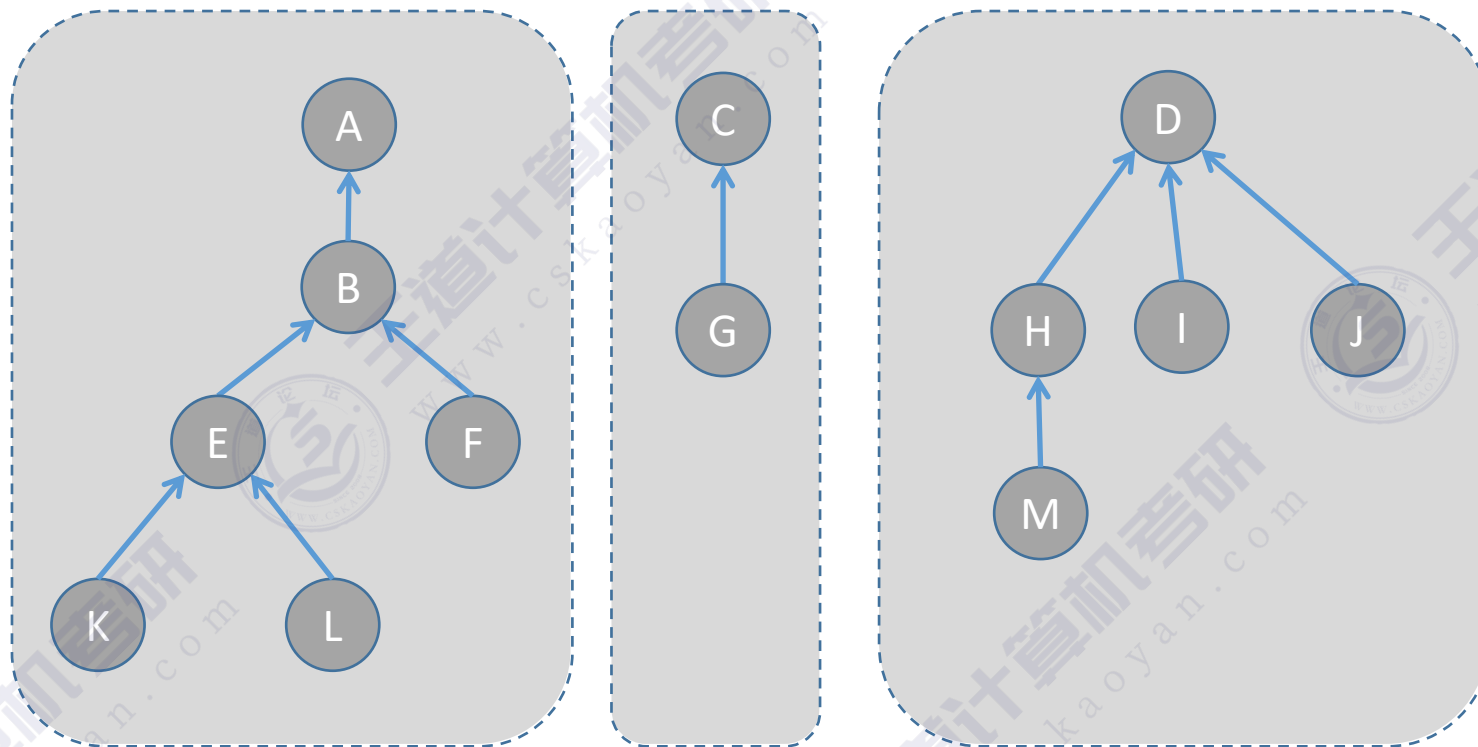
2009	2010	2011	2012	2013	2014	2015	2016	2017
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频: NULL

并查集



缓缓地回忆起过去



数据元素

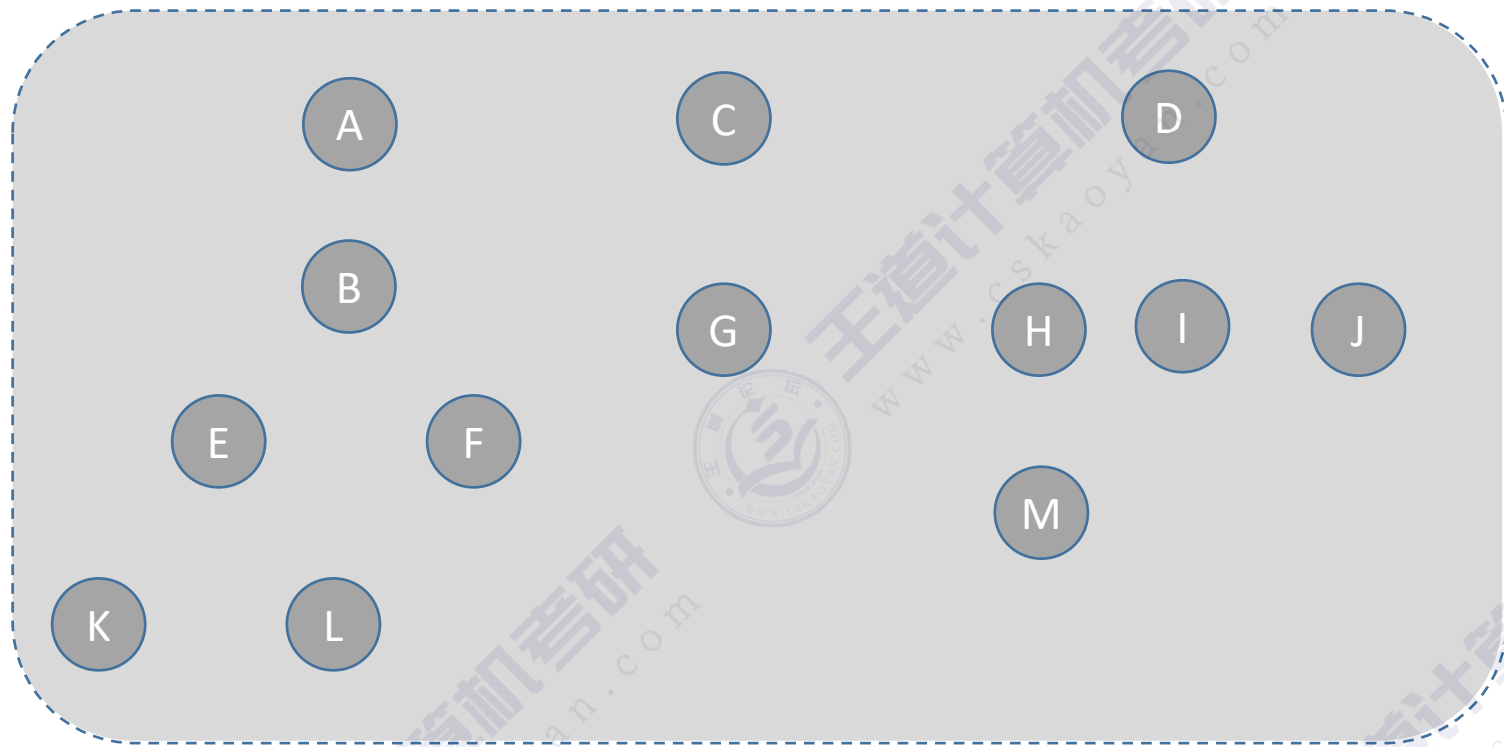
数组下标

S[]

A	B	C	D	E	F	G	H	I	J	K	L	M
0	1	2	3	4	5	6	7	8	9	10	11	12
-6	0	-2	-5	1	1	2	3	3	3	4	4	7

本质上与“树的双亲表示法”相同，每个集合对应一棵树

并查集——定义&初始化



```
#define SIZE 13
int UFSets[SIZE];    //集合元素数组

//初始化并查集
void Initial(int S[]){
    for(int i=0;i<SIZE;i++){
        S[i]=-1;
    }
}
```

数据元素
数组下标

	A	B	C	D	E	F	G	H	I	J	K	L	M
	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

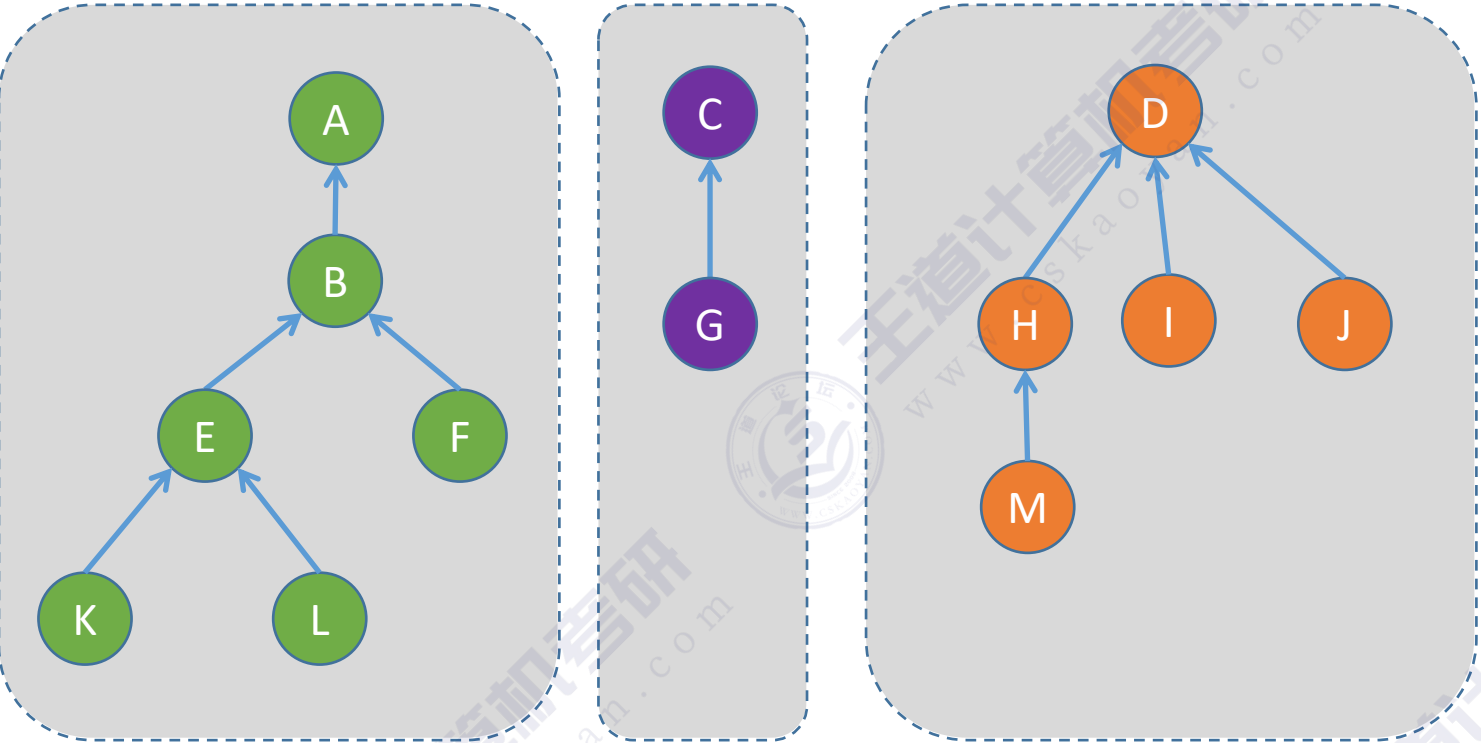
用一个数组 S[] 即可表示“集合”关系

注意：Find操作的效率与树高相关。最坏情况下，n个元素组成的树高为O(n)

并查集——并&查操作

```
//Find “查”操作，找x所属集合（返回x所属根结点）
int Find(int S[],int x){
    while(S[x]>=0) //循环寻找x的根
        x=S[x];
    return x; //根的s[]小于0
}
```

```
//Union “并”操作，小树合并到大树
void Union(int S[],int Root1,int Root2){
    if(Root1==Root2) return;
    if(S[Root2]>S[Root1]) { //Root2结点数更少
        S[Root1] += S[Root2]; //累加结点总数
        S[Root2]=Root1; //小树合并到大树
    } else {
        S[Root2] += S[Root1]; //累加结点总数
        S[Root1]=Root2; //小树合并到大树
    }
}
```



数据元素	A	B	C	D	E	F	G	H	I	J	K	L	M
数组下标	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-6	0	-2	-5	1	1	2	3	3	3	4	4	7

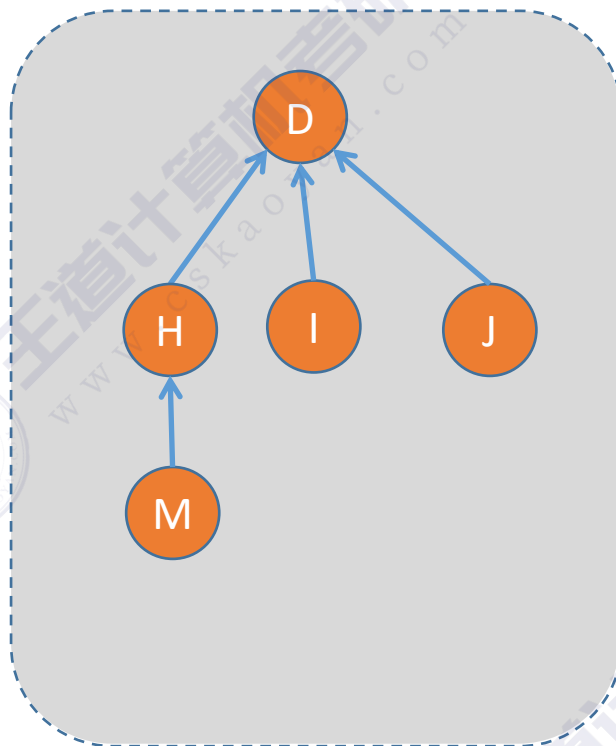
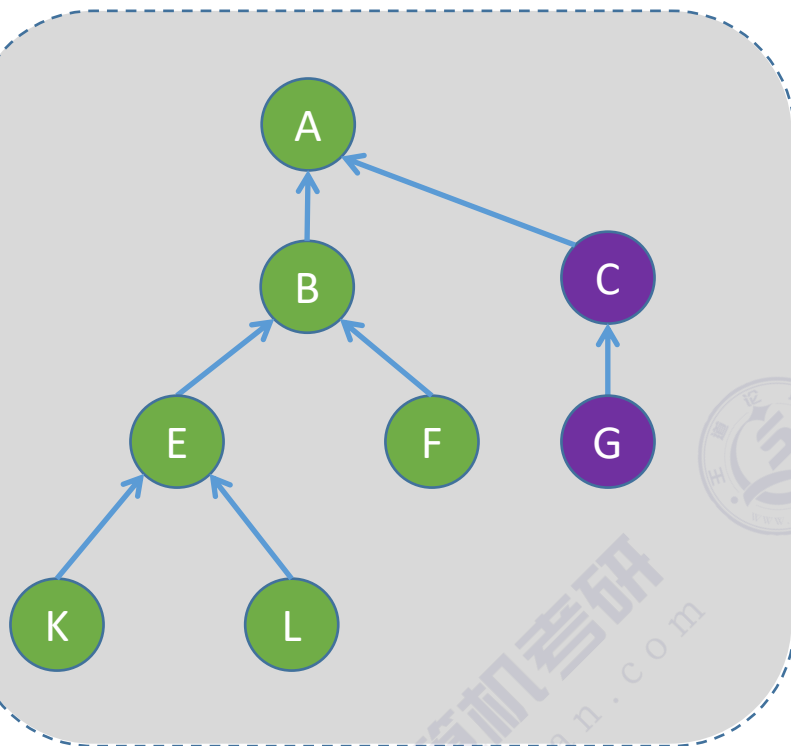
①用根节点的绝对值表示树的结点总数

注意：“小树并入大树”可以使得树高不超过 $O(\log_2 n)$

并查集——并&查操作

```
//Find “查”操作，找x所属集合（返回x所属根结点）
int Find(int S[],int x){
    while(S[x]>=0)           //循环寻找x的根
        x=S[x];
    return x;                //根的s[]小于0
}
```

```
//Union “并”操作，小树合并到大树
void Union(int S[],int Root1,int Root2){
    if(Root1==Root2) return;
    if(S[Root2]>S[Root1]) { //Root2结点数更少
        S[Root1] += S[Root2]; //累加结点总数
        S[Root2]=Root1; //小树合并到大树
    } else {
        S[Root2] += S[Root1]; //累加结点总数
        S[Root1]=Root2; //小树合并到大树
    }
}
```



数据元素
数组下标

数据元素	A	B	C	D	E	F	G	H	I	J	K	L	M
数组下标	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-8	0	0	-5	1	1	2	3	3	3	4	4	7

①用根节点的绝对值表示树的结点总数

②Union操作，让小树合并到大树

并查集怎么考？——选择题



举个例子？

可能出现的考法：

下列关于并查集的说法中，正确的是？（ ）

下列关于并查集的说法中，错误的是？（ ）

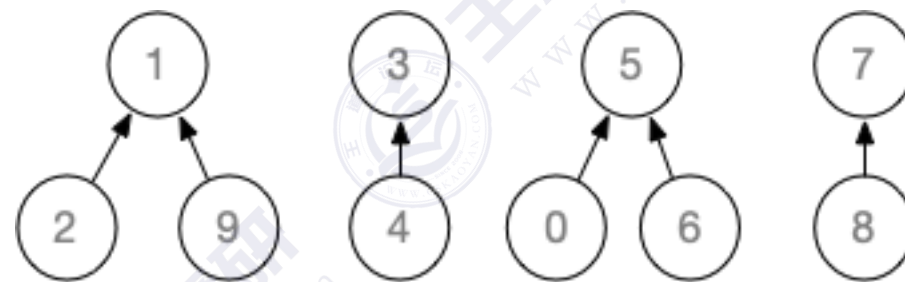
- ☒ • 并查集是孩子表示法存储的树
- ☒ • 并查集是双亲表示法存储的树
- ☒ • 并查集的查找效率和树的高度无关
- ☒ • 若并查集中包含 n 个元素，则Find操作最好时间复杂度为 $O(n)$
- ☒ • 为了提高并查集的查找效率，在进行Union操作时，可以让结点更少的树作为结点更多的树的子树
- ☒ • 为了提高并查集的查找效率，可以采用“压缩路径”的方式优化Find操作
- ☒ • 并查集可以用于实现克鲁斯卡尔算法求最小生成树
- ☒ • 并查集用于判断无向图的连通性
- ☒ • 并查集用于判断无向图中是否有环
- ☒ • 并查集可以用于迪杰斯特拉算法求最短路径

并查集怎么考？——应用题（画图）

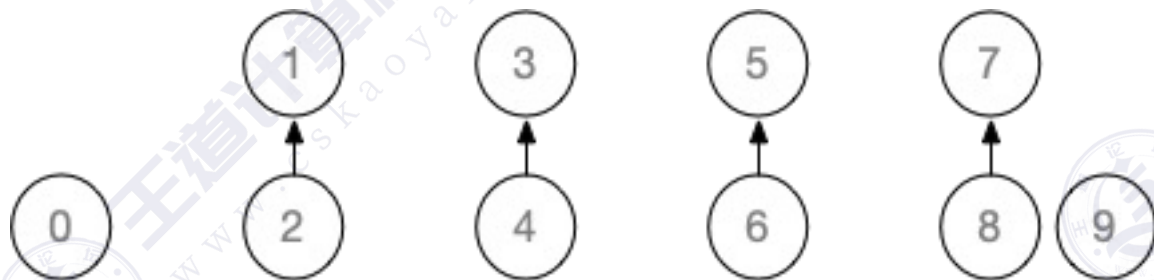
假设初始长度为 10（0 ~ 9）的并查集，按 1-2、3-4、5-6、7-8、0-5、1-9、9-3、8-0、4-6 的顺序进行Union操作



初始状态👉



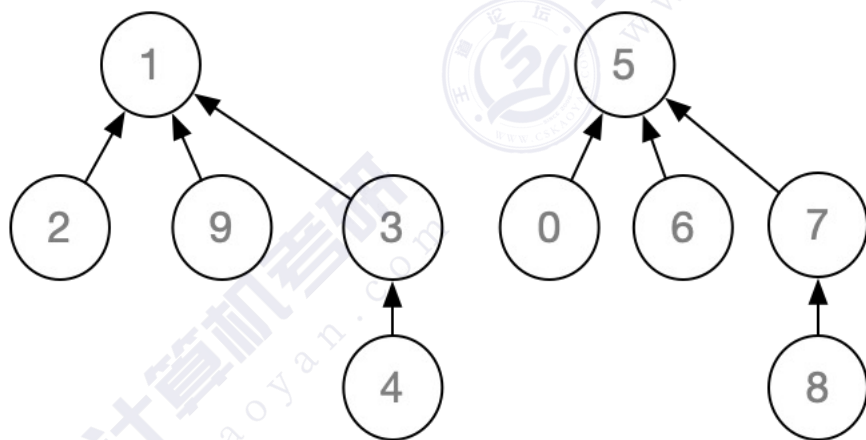
Union: 0-5、1-9👉（小树合并到大树）



Union: 1-2、3-4、5-6、7-8👉

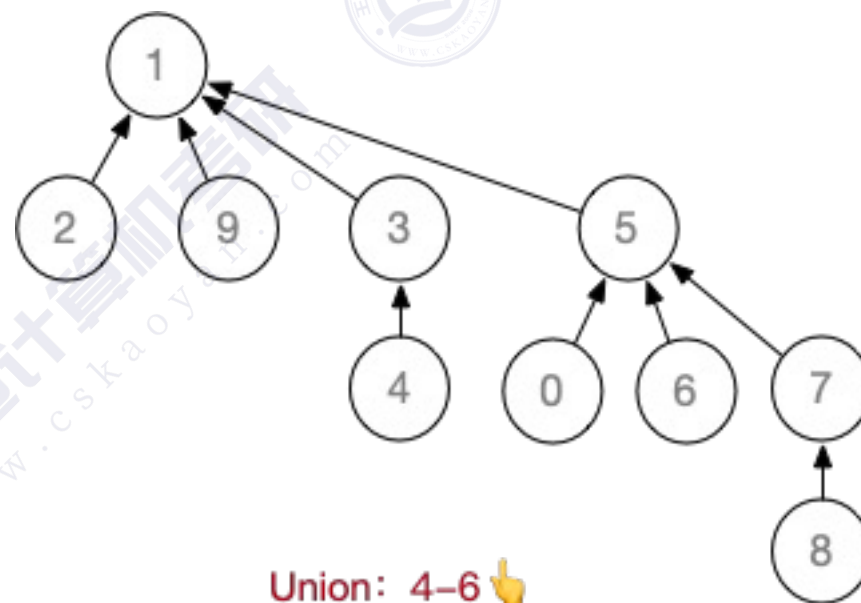
并查集怎么考？——应用题（画图）

假设初始长度为 10（0 ~ 9）的并查集，按 1-2、3-4、5-6、7-8、0-5、1-9、9-3、8-0、4-6 的顺序进行Union操作



Union: 9-3、8-0 🙌

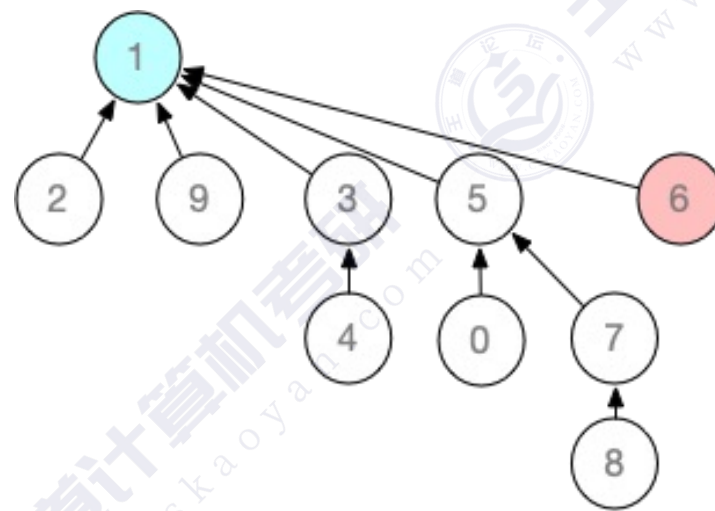
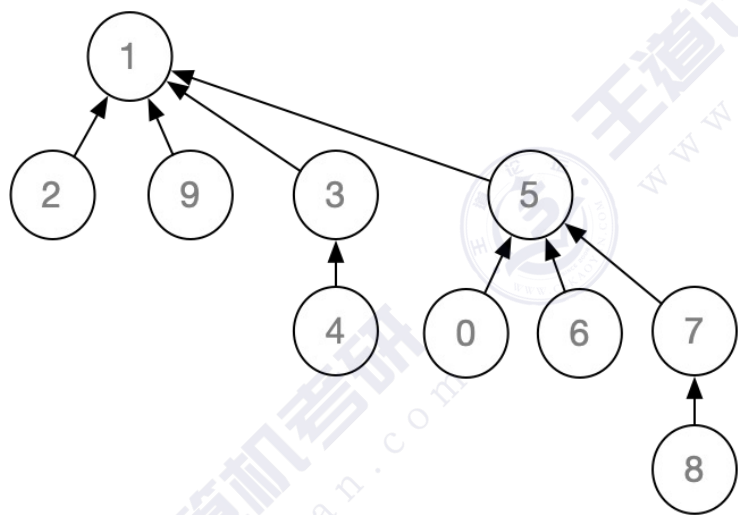
(先找要Union操作的两个集合的根，再将小树合并到大树)



Union: 4-6 🙌

并查集怎么考？——应用题（画图）

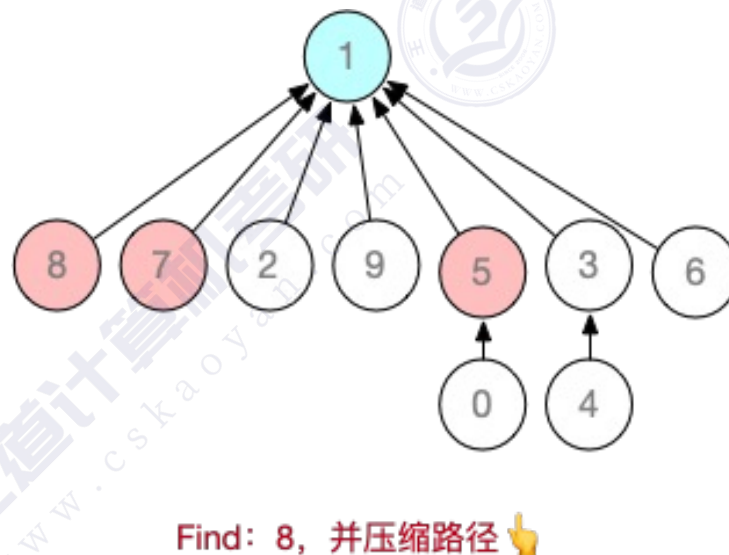
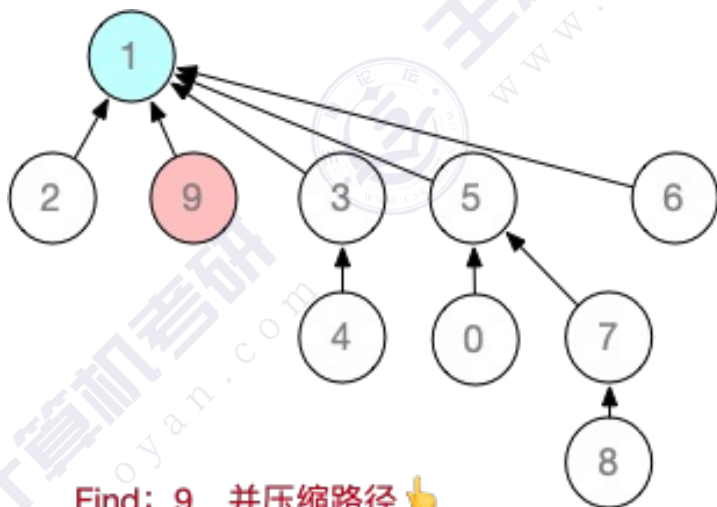
依次对 6、9、8 执行 Find 操作，并完成压缩路径



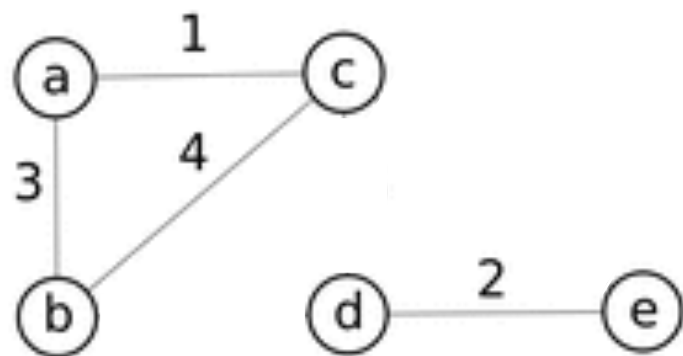
Find: 6, 并压缩路径 🖱️

并查集怎么考？——应用题（画图）

依次对 6、9、8 执行 Find 操作，并完成压缩路径



拓展1：用并查集判断图的连通分量个数



遍历各条边，有边相连的两个顶点一定是连通的，将两个顶点所属集合“并”为一个集合

处理完所有边，即可将图划分为若干个连通分量

用邻接矩阵表示无向图的边：

$g[5][5]$

	a	b	c	d	e
a	0	3	1	0	0
b	3	0	4	0	0
c	1	4	0	0	0
d	0	0	0	0	2
e	0	0	0	2	0

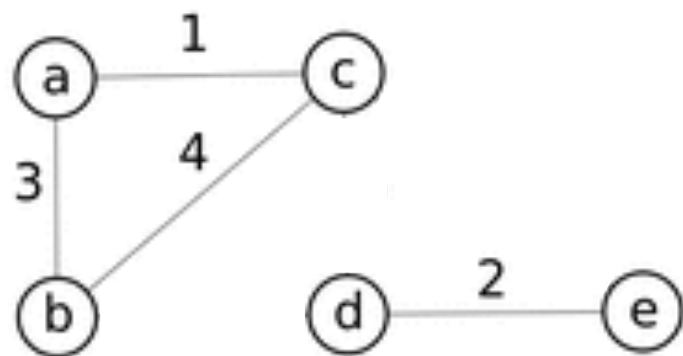
0 1 2 3 4

并查集 $S[]$

--	--	--	--	--

注：若连通分量个数=1，说明这个无向图“连通”

拓展2：用并查集判断图是否有环



遍历各条边，有边相连的两个顶点一定是连通的，将两个顶点所属集合“并”为一个集合

已经连通的子图中，但凡再多出一条边，这个子图就一定有环

用邻接矩阵表示无向图的边：

$g[5][5]$

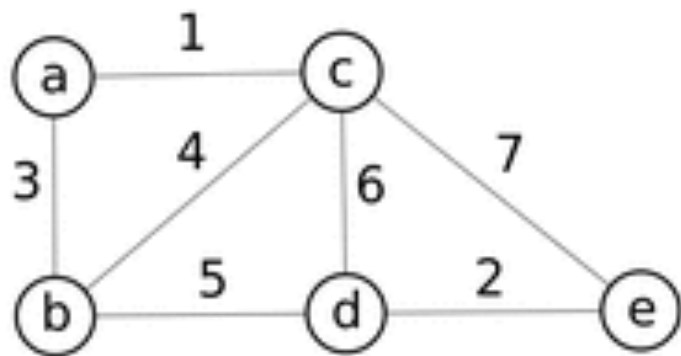
	a	b	c	d	e
a	0	3	1	0	0
b	3	0	4	0	0
c	1	4	0	0	0
d	0	0	0	0	2
e	0	0	0	2	0

0 1 2 3 4

并查集 $S[]$

--	--	--	--	--

拓展3：用并查集实现Kruskal 算法



用邻接矩阵表示无向图的边：

	a	b	c	d	e
a	0	3	1	0	0
b	3	0	4	5	0
c	1	4	0	6	7
d	0	5	6	0	2
e	0	0	7	2	0

Kruskal算法，初始：各条边按权值递增排序

(a,c) (d,e) (a,b) (b,c) (b,d) (c,d) (c,e)

初始化并查集，按权值递增次序处理各条边：

通过 Find 确定一条边所连接的两个顶点是否属于同一个集合
如果不属于同一集合，则将这条边加入生成树，并将两个点所属集合 Union